

ASEGURAMIENTO CALIDAD SOFTWARE
Avance Proyecto Final
Sistema de Gestión de Inventarios con QAS

Descripción General:

El objetivo del proyecto es desarrollar un sistema de gestión de inventarios para una pequeña empresa, que cubra todas las etapas del ciclo de vida del aseguramiento de la calidad del software (QAS). El sistema debe incluir funcionalidades básicas de gestión de productos, control de stock y más, además de cumplir con altos estándares de calidad, seguridad y usabilidad. El proyecto debe ser realizado en grupos de máximo dos estudiantes.

Funcionalidades del Sistema de Gestión de Inventarios

1. Gestión de Productos

- a. **Agregar Producto:** Permitir a los usuarios agregar nuevos productos al inventario, incluyendo detalles como nombre, descripción, categoría, precio y cantidad inicial.
- b. **Editar Producto:** Permitir la edición de la información de un producto existente.
- c. **Eliminar Producto:** Permitir la eliminación de productos del inventario.
- d. **Visualizar Productos:** Mostrar una lista de todos los productos en el inventario con opciones de búsqueda y filtrado.

2. Control de Stock

- a. **No requerido**

3. Integración con Otros Sistemas

- a. **API de Integración:** Proporcionar una API para integrar el sistema con otras aplicaciones (por ejemplo, sistemas de contabilidad o puntos de venta).

4. Interfaz de Usuario Amigable

- a. **No requerido**

Roles y Niveles de Acceso

1. Administrador

- a. **Descripción:** Tiene acceso completo a todas las funcionalidades del sistema.
- b. **Permisos:**
 - i. **Gestión de Productos:** Agregar, editar, eliminar y visualizar productos.
 - ii. **Control de Stock:** Actualizar stock y visualizar historial de movimientos.

2. Empleado

a. No requerido

3. Usuario Invitado/Cliente

a. No requerido

Funcionalidad	Administrador	Empleado	Invitado
Gestión de Productos	CRUD	CRUD	R
Control de Stock	CRUD	CRUD	-
API de Integración	JWT	-	-

Etapas del Ciclo de Vida del Aseguramiento de la Calidad del Software (QAS)

1. Planificación y Gestión de Proyectos:

- a. Plan de Proyecto: Definir claramente el alcance, los objetivos, los entregables y el cronograma del proyecto.
- b. Gestión de Tareas: Utilizar herramientas como Jira, Trello o Asana para gestionar las tareas del proyecto y realizar un seguimiento del progreso.
- c. Gestión de Riesgos: Identificar y documentar los posibles riesgos y planificar estrategias de mitigación.

2. Pruebas de Calidad:

- a. Pruebas de Seguridad: No requerido
- b. Pruebas de Usabilidad: No requerido
- c. Pruebas de Compatibilidad: Asegurar que el sistema funcione correctamente en diferentes navegadores y dispositivos.
- d. Pruebas de Regresión: No requerido
- e. Pruebas de Estrés: No requerido
- f. Pruebas de Aceptación: Implementar pruebas de aceptación basadas en Cucumber para asegurar que el sistema cumple con los requisitos del usuario.
- g. Pruebas de Navegadores: Utilizar Playwright para realizar pruebas automatizadas en diferentes navegadores y dispositivos, asegurando la compatibilidad.

3. Documentación:

- a. Documentación de Requisitos: Crear un documento detallado de requisitos funcionales y no funcionales.
- b. Documentación Técnica: No requerido
- c. Guía de Pruebas: Documentar los casos de prueba, los resultados y cualquier defecto encontrado.

4. Revisión y Validación:

- a. Revisiones de Código: No requerido
- b. Validación de Requisitos: No requerido

5. Integración Continua y Despliegue Continuo (CI/CD):

- a. Pipeline de CI/CD: *No requerido*
- b. Entorno de Pruebas: *No requerido*
- c. Contenedorización: Utilizar herramientas como Docker o Podman para crear contenedores del sistema que faciliten la consistencia entre diferentes entornos de desarrollo, prueba y producción.
- d. Migración de Base de Datos: Implementar herramientas como Flyway o Liquibase para gestionar las migraciones de bases de datos de manera segura y automatizada.

6. Monitoreo y Mantenimiento:

- a. Monitoreo en Producción: *No requerido*
- b. Observabilidad: *No requerido*
- c. Mantenimiento y Actualizaciones: Planificar un ciclo regular de mantenimiento y actualizaciones para asegurar que el sistema permanezca seguro y funcional.

7. Gestión de Calidad:

- a. Indicadores de Calidad: Definir métricas de calidad del software (como cobertura de pruebas, densidad de defectos, tiempo de respuesta) y realizar un seguimiento de estas métricas.
- b. Mejora Continua: *No requerido*

8. Evaluación Post-Implementación:

- a. Revisión Post-Mortem: *No requerido*

Tecnologías Sugeridas:

- Backend: Spring Boot, Quarkus, Django, Node.js, Next.js
- Auditoría: Hibernate Envers
- Frontend: Hilla, Vaadin Flow, React.js, Vue.js, Angular, JHipster
- Base de Datos: MySQL, PostgreSQL, MariaDB
- Auditoría: Hibernate Envers
- Autenticación y Autorización: OAuth2, JWT.
- Pruebas: JUnit/TestNG, Selenium, Cucumber, JMeter, Playwright, Microcks
- CI/CD: GitHub Actions, Jenkins, GitLab CI.
- Contenedorización: Docker, Podman.
- Migración de Base de Datos: Flyway, Liquibase.
- Monitoreo y Observabilidad: Prometheus, Grafana, Open Telemetry

Conclusión:

Este mandato proporciona una guía integral para el desarrollo de un sistema de gestión de inventarios con un enfoque en el aseguramiento de la calidad del software. Al seguir estas directrices, se garantizará que el sistema no solo sea funcional y eficiente, sino también seguro, confiable y fácil de usar. El proyecto debe ser realizado en grupos de máximo dos estudiantes, promoviendo el trabajo en equipo y la colaboración.

La valoración de cada miembro del equipo se tomará en cuenta considerando los commits realizados en GitHub. Asegúrense de que ambos miembros del equipo contribuyan de manera equitativa al proyecto.